

Tiled Tutorial

Draw a map with objects in Tiled, export it as JSON, and load it into the engine you already built — replacing a flat background image with a real, editable level format.

Editor: Tiled (mapeditor.org) – free, open source

Companion code: `engine/tilemap.py` · `data/level1.tmj` ·
`scenes/tiled_play_scene.py`

Try it now: `python main_tiled_demo.py`

Table of Contents

1	Why Move Off a Flat Image	what you gain from Tiled
2	Installing Tiled	one-time, then fully offline
3	Your First Map	the New Map wizard, explained
4	Keeping Your Existing Art	the Image Layer
5	Drawing Objects	spawn points, names, custom properties
6	Saving as JSON	.tmj, and what's actually in it
7	Loading It In Your Engine	the real, tested code
8	Coordinate Gotchas	point vs. rectangle vs. tile objects
9	Exercises	five hands-on extensions
10	Tiled Troubleshooting	offline-friendly fixes
11	Porting Into M151_Engine	replacing the empty level_loader.py stub

1 • Why Move Off a Flat Image

Right now your maps are one big painted image

(`assets/images/map/m151_practice_stage.png`) with spawn positions hand-typed as pixel offsets in a separate JSON file (`data/enemies.json`). That works, but it has a real limitation: **the map file and the placement data live in two unrelated places**, and lining them up means guessing pixel coordinates, running the game, and adjusting numbers by trial and error.

Tiled fixes this by letting you draw directly on top of your map and place named, typed, propertied **objects** exactly where you want them, visually. Export that as JSON and you get one file that describes both the map and everything that spawns on it — `data/enemies.json` 's job, absorbed into the same file as the map itself.

WHAT YOU'RE NOT GIVING UP

You don't have to redraw your background art as tiles. Tiled supports an **Image Layer** that displays one big picture exactly like you're doing now — Chapter 4 covers this directly. Objects (spawn points, collision zones) go in a separate layer on top. You can adopt Tiled today without touching your existing art at all.

2 • Installing Tiled

Tiled is free, open source, and available for Windows, macOS, and Linux from **mapeditor.org**. This is the one step in this whole tutorial that needs an internet connection — download the installer (or the portable/AppImage build on Linux) once, and everything after that works completely offline. There's no license key, no account, no online-only features.

NOTE

This tutorial describes Tiled's stable 1.x workflow — new-map creation, layers, objects, and JSON export. These have been stable for years and are extremely unlikely to have moved to different menus even if your installed version differs slightly from the one this was written against.

3 • Your First Map

`File ▶ New ▶ New Map...` opens a dialog with a few settings that matter:

Setting	What to pick	Why
Orientation	Orthogonal	A plain top-down grid — matches this engine's camera, which has no concept of isometric or hexagonal projection.
Tile layer format	Base64 (zlib compressed), or leave default	Only matters if you paint tile layers later (Exercise 5). Doesn't affect image layers or objects at all.
Tile render order	Right Down (default)	Irrelevant for image layers and objects; only affects tile-layer draw order.
Map size	Fixed, not Infinite	Leave "Infinite map" unchecked. Infinite maps store tile data in chunks, which is real added complexity this tutorial's loader deliberately doesn't handle (see <code>engine/tilemap.py</code> 's check for this).
Tile width / height	32 × 32	Matches this project's sprite sheet cell size (<code>config.SPRITE_W / SPRITE_H</code>). This only sets the editor's grid snapping — it does not resize your background image.
Map width / height (in tiles)	Match your background image's pixel size ÷ tile size	E.g. a 1920×5760px background at 32px tiles needs a 60×180 tile map, so the canvas is exactly big enough to contain it.

WORKED EXAMPLE

`assets/images/map/m151_practice_stage.png` is 1920×5760 pixels. At 32×32 tiles, that's **60 tiles wide, 180 tiles tall** — exactly the numbers used in this tutorial's sample `data/level1.tmj`.

4 · Keeping Your Existing Art: the Image Layer

With the new map open, remove the default empty tile layer (right-click it in the Layers panel ► `Remove Layer`), then add an image layer instead:

- 1 `Layer ► New ► Image Layer`
- 2 In the Properties panel (bottom-right by default), click the `Image` field's `...` button and browse to your existing map art — e.g. `assets/images/map/m151_practice_stage.png`.
- 3 Leave the layer's `x / y` offset at 0, 0 so the image's top-left corner lines up with world-space (0, 0) — the same convention `Entity.world_x / world_y` already uses.
- 4 Rename the layer to something you'll recognize in code later, e.g. `background` — the loader looks layers up by name.

Your existing artwork now shows up in the Tiled canvas exactly as painted, and you can draw objects directly on top of it, visually lined up against the real background — something pixel-guessing in a JSON file could never give you.

5 • Drawing Objects

Add a layer to hold your spawn points:

- 1 `Layer ▶ New ▶ Object Layer`, name it `enemies` (matching whatever name your loading code will ask for).
- 2 Select the **Insert Point** tool (a small crosshair icon in the toolbar, or press `P` ← check your version's shortcut in `View ▶ Toolbars` if unsure) and click once on the canvas, right where you want an enemy to spawn. A point object needs no width or height — it's just a location.
- 3 With the new object selected, use the Properties panel to set:
 - **Name** — a label for this specific object, e.g. `alpha`.
 - **Class** (older Tiled calls this "Type") — a category shared by similar objects, e.g. `enemy_spawn`.
- 4 Repeat for every enemy you want to place. Use the regular Select tool afterward to drag any point to a new spot — no coordinates to retype by hand.

Custom Properties

With an object selected, find the **Custom Properties** section of the Properties panel and click the `+` button to add one. This is how you attach the same data `data/enemies.json` already stores per enemy — just typed into fields instead of JSON syntax:

Property name	Type	Example value
<code>trail</code>	string	<code>patrol</code> , <code>bounce</code> , or <code>zigzag</code>
<code>speed</code>	int	<code>180</code>

NOTE

These property names aren't special to Tiled — they're just labels *you* chose to match what `Enemy.__init__` already expects (`trail=`, `speed=`). Tiled has no idea what an "Enemy" is; it just stores whatever named properties you type, as data, for your code to read back later.

6 · Saving as JSON

`File ▶ Export As...`, choose **JSON map files (*.tmj)** as the file type. (Some Tiled versions let you `Save As` directly with a `.tmj` extension instead of a separate export step — either produces the same format.) Save it as `data/level1.tmj` in your project.

What's actually inside the file

Open the exported `.tmj` in a text editor — it's plain JSON. The parts this engine's loader cares about:

```
{
  "width": 60, "height": 180, "tilewidth": 32, "tileheight": 32,
  "layers": [
    {
      "type": "imagelayer",
      "name": "background",
      "image": "../assets/images/map/m151_practice_stage.png",
      "x": 0, "y": 0
    },
    {
      "type": "objectgroup",
      "name": "enemies",
      "objects": [
        {
          "name": "alpha", "type": "enemy_spawn",
          "x": 1220, "y": 2880,
          "point": true,
          "properties": [
            { "name": "trail", "type": "string", "value": "patrol" },
            { "name": "speed", "type": "int", "value": 180 }
          ]
        }
      ]
    }
  ]
}
```

GOTCHA: PROPERTIES ARE A LIST, NOT A DICT

Every object's `properties` field is a JSON array of `{"name", "type", "value"}` objects — not a plain `{"trail": "patrol"}` dict, even though that's what you'd probably reach for by hand. Loading code has to flatten this list into a dict itself; engine/tilemap.py's `_simplify_object()` does exactly that:

```
properties = {p["name"]: p["value"] for p in obj.get("properties", [])}
```

GOTCHA: IMAGE PATHS ARE RELATIVE TO THE .TMJ FILE

"image": "../assets/images/map/..." is relative to *wherever level1.tmj itself is saved*, not your current working directory or the project root. If you move the .tmj file without moving the image (or vice versa), Tiled will show a broken-image icon, and your loader needs to resolve the path the same way Tiled does — relative to the map file's own folder. That's exactly what `TiledMap.image_layer()`'s `self._path.parent / layer["image"]` does.

7 • Loading It In Your Engine

Real, tested code – try it now: `python main_tiled_demo.py`

Three new files make this work, and none of them touch `engine/entity.py`, `Player`, `Enemy`, `Missile`, `EntityManager`, or `CollisionSystem` at all:

`engine/tilemap.py` — the generic loader

```
class TiledMap:
    def __init__(self, tmj_path):
        with open(tmj_path) as f:
            self._data = json.load(f)
        if self._data.get("infinite"):
            raise ValueError("infinite maps aren't supported by this loader")
        ...

    def image_layer(self, name=None):
        # returns {"path": Path, "x": int, "y": int}

    def object_layer(self, name):
        # returns [{"name", "type", "x", "y", "properties"}, ...]
```

This file lives in `engine/` because it doesn't know or care what an "Enemy" is — same rule as every other engine system in this project (see the main usage guide's Chapter 1). It just turns a `.tmj` file into plain Python data.

`scenes/tiled_play_scene.py` — where the data becomes entities

```
tiled_map = TiledMap(LEVEL_TMJ)
background = tiled_map.image_layer("background")
self.map_surface = self.resources.load_image(background["path"])

for spec in tiled_map.object_layer("enemies"):
    enemy = Enemy(
        spec["name"], spec["x"], spec["y"], self._enemy_frames,
        trail=spec["properties"].get("trail", "patrol"),
        speed=spec["properties"].get("speed", config.ENEMY_SPEED),
    )
    self.entities.add(enemy, group="enemies")
```

WHY THIS PROVES THE ARCHITECTURE WORKS

`TiledPlayScene` is nearly line-for-line identical to `PlayScene` after this point — `handle_event`, `update`, `draw`, missile-firing, and collision are completely unchanged. Swapping the entire source of level data (JSON file with offsets →

Tiled file with absolute positions) only touched the handful of lines that build the initial entity list. That's the payoff of `EntityManager` and `Entity` not knowing where spawn data comes from.

8 • Coordinate Gotchas

Tiled objects always store coordinates in **pixels** (never tile units), which is convenient — no conversion math needed before handing an object's `x / y` straight to an `Entity`. But WHERE that `x/y` points depends on the object's shape:

Object type	What (x, y) means
Point (what this tutorial uses)	The point itself — exactly the location you clicked.
Rectangle	The rectangle's top-left corner — same convention <code>Entity.world_x / world_y</code> already uses.
Tile object (an object with a sprite picked from a tileset)	The bottom-left corner — a historical Tiled quirk, different from every other object type. If you ever place tile objects, subtract the object's height from <code>y</code> before using it as a top-left position.

GOTCHA: ALSO WATCH THE OFFSET-FROM-CENTER VS. ABSOLUTE MISMATCH

`data/enemies.json` stores `spawn_x_from_center` — an offset that `JSONSpawner` adds to `map_w / 2`. Tiled objects store **absolute world pixels** directly, with no such offset. `TiledPlayScene` uses `spec["x"] / spec["y"]` straight, unmodified — mixing the two conventions (e.g. accidentally adding `map_w/2` to a Tiled coordinate) would place every enemy in the wrong spot.

9 · Exercises

- 1 **Add a fifth enemy purely in Tiled.** Open `data/level1.tmj` in Tiled, place a new point object in the `enemies` layer, give it a name and `trail / speed` properties, export, and rerun `main_tiled_demo.py`. No Python changes at all.
- 2 **Add an obstacle layer.** Create a second object layer named `obstacles`, draw a few Rectangle objects over parts of the map the player shouldn't walk through, and write a small `Obstacle(Entity)` (static, `update()` does nothing) that `TiledPlayScene` spawns from that layer and checks the player against with `collision.check_groups`.
- 3 **Compare both level sources side by side.** Run `python main.py` (JSON-driven) and `python main_tiled_demo.py` (Tiled-driven) back to back — the enemy layout should feel identical, because `data/level1.tmj` was hand-built to mirror `data/enemies.json` exactly.
- 4 **Rename a layer and watch it fail loudly.** Rename the `enemies` object layer to something else in Tiled, re-export, and rerun the demo. Read the `KeyError` `engine/tilemap.py` raises — this is deliberate: a missing layer should be a clear, immediate error, not enemies silently failing to spawn.
- 5 **Try a real tileset** (bigger step). Instead of an Image Layer, add a Tileset (`Map ▶ New Tileset...`) built from small reusable tile images, and paint a Tile Layer with them. This is real terrain-authoring, not covered by `engine/tilemap.py` as written — see the main usage guide's Chapter 9 ("Where To Go Next") for what a tile-layer renderer would add.

10 · Tiled Troubleshooting

Symptom	Likely cause	Fix
Image layer shows a broken/missing image icon	The <code>.tmj</code> and the image file moved relative to each other since the layer was created.	Re-browse the Image field to the file's current location, then re-save/export.
Objects don't appear in the exported JSON at all	The object layer is hidden (eye icon off in the Layers panel) — hidden layers still export, so this usually means you're looking at the wrong layer, or objects were accidentally placed on the image layer instead of an object layer.	Check the Layers panel: objects must live under a layer whose type is Object Layer, not Image Layer or Tile Layer.
<code>KeyError</code> from <code>TiledMap.object_layer()</code>	The name passed in code doesn't match the layer's name in Tiled exactly (case-sensitive).	Open the <code>.tmj</code> and check the layer's <code>"name"</code> field, or rename the layer in Tiled to match your code.
Custom property doesn't show up in <code>spec["properties"]</code>	The property was added to the wrong object, or to the layer itself instead of an individual object.	Click directly on the point/object (not the layer name) in the Layers panel before adding the property.
Map looks enormous / tiny in the Tiled canvas	Map width/height (in tiles) doesn't match the background image's actual pixel size $\div$ tile size.	Recompute: <code>tiles_wide = image_width_px / tile_width</code> , and fix it under <code>Map ► Resize Map...</code> .
<code>ValueError: infinite maps aren't supported</code>	"Infinite map" was left checked in the New Map dialog.	Start a fresh map with it unchecked — there's no toggle to fix this after the fact without recreating the map.

11 · Porting Into M151_Engine

The Study Guide's Migration Map (Section 9) already flagged

`gameplay/level/level_loader.py` and `gameplay/level/level1.json` in your main project as **both empty** — a deliberate green field for exactly this. Now that you have a working Tiled pipeline here, the recommendation is direct:

- 1 Author your real level in Tiled the same way as this tutorial — Image Layer for the existing background art, Object Layer(s) for enemies (and anything else you need placed).
- 2 Save it as `gameplay/level/level1.tmj` instead of the empty `level1.json` — a `.tmj` file is JSON, just with a more conventional extension for Tiled tooling to recognize.
- 3 Port `engine/tilemap.py` into `gameplay/level/level_loader.py` unchanged — it has no dependency on anything game-specific, same as every other engine-layer file in this project.
- 4 Wire it into whatever Scene/Game structure you built during Week 2 of the Study Guide, following `scenes/tiled_play_scene.py` here as the template for "read the file, build entities from its object layer."

M151 Tiled Tutorial — every code sample in this document is real and tested: `python main_tiled_demo.py` runs it. Draw your own level whenever you're ready, drop it in as `data/level1.tmj`, and everything else in this engine already knows what to do with it.